INDEPENDENT STUDY PROJECT PROPOSAL

Course Enrollment: COS 497 – Independent Study (3 Credits)
Course Equivalence: COS 360 – Programming Languages
Term: Summer Session II 2026
Student: Vivek Reddy Bhimavarapu (vivek.bhimavarapu@maine.edu)

1) Learning Objectives

By completing this accelerated independent study, I will achieve the core learning outcomes specified by the standard COS 360 curriculum:

- **Paradigm Evaluation:** Program in and contrast different language paradigms (Imperative/Object-Oriented, Functional, and Scripting) using Python as a multi-paradigm vehicle.
- **Formal Syntax & Semantics:** Describe the methods of recognition for regular and context-free languages, including how code constructs are parsed into abstract syntax trees (ASTs) to dictate control flow.
- **Functional Concepts:** Explain the functional alternative to state-based imperative paradigms and write algorithms relying strictly on functional constraints (immutability, recursion, and higher-order functions).
- **Design Trade-offs:** Identify and explain the structural differences, design influences, and implementation trade-offs (static vs. dynamic typing, compilation vs. interpretation) across major language domains.
- **Concurrency Models:** Understand concurrent programming synchronization primitives and how different language runtimes manage asynchronous execution paradigms.

2) Activities

- **Phase I: Formal Language Theory & Parsing (Weeks 1–2)**
    - Study regular and context-free grammar constraints, Lexical Analysis, and Syntactic Analysis using chapters from the course text.
    - ***Activity****:* Implement a lightweight, from scratch lexer and recursive descent parser in Python that takes a basic mathematical text expression, parses it, and outputs its Abstract Syntax Tree (AST) representation.
- **Phase II: The Functional Paradigm via Multi-Paradigm Python (Weeks 3–5)**
    - Deep dive into pure functional programming concepts—including immutability, first-class/higher-order functions, lambda calculus, and pure recursion—by constraining Python strictly to its functional features.
    - ***Activity:*** Complete a comprehensive suite of algorithmic and data-manipulation exercises using purely functional Python (forgoing standard for/while loops and variable re-assignments in favor of map, filter, reduce, and recursion).
- **Phase III: Comparative Analysis & Concurrency (Weeks 6–7)**
    - Examine language design trade-offs, type systems, and concurrency primitives (such as threads, locks, and asynchronous event loops).
    - *Activity:* Author a formal technical monograph comparing three distinct paradigms (Imperative Python, Functional Python, and Asynchronous TypeScript) across memory management, type safety, and parallel execution.

3) Resources

This project is entirely self funded and relies on open source software, will not require budget or specialized hardware from the university.

- **Primary Bibliography (Self-Provided):**
    - *Programming Languages Principles and Practices (3rd Edition)* by Louden, K. & Lambert K. (The official department course text).
    - *Functional Programming in Python* by David Mertz (For the functional programming module).
- **Software & Equipment:**
    - Personal laptop utilizing standard open-source runtimes (Python 3.x, Node.js/TypeScript, and Docker for environment isolation).
    - GitHub repository for project hosting and rigorous version control tracking.
- **People:**
    - **Faculty Sponsor:** Serves strictly in an evaluative and advisory capacity, requiring no active lecturing or lab setup.

4) Evaluation

To streamline administration for the faculty sponsor over the summer, the standard syllabus evaluation components (quizzes, exams, and homework) are condensed into definitive, project-based milestones. The 100% course grade will be mapped directly to the following artifacts:

Evaluation Component	**Weight**	**Deliverable Details**
Code Artifact A *(Parser)*	**30%**	**Due Week 3:** Custom lexer and expression parser script in Python that generates an AST, validated by a clean unit test suite.
Code Artifact B *(Functional Portfolio)*	**30%**	**Due Week 5:** A repository of Python source files solving algorithmic problems structured *strictly* under functional constraints (relying on pure functions, higher-order functions, and recursion).
Course Project & GitHub	**20%**	**Due Week 7:** Integration of the code portfolio into a production-ready, open-ended repository explicitly utilizing version control with Git and hosted on GitHub.

Final Technical Monograph	**20%**	**Due Week 7:** An 8-page final comparative analysis paper synthesizing language design trade-offs, parsing theory, and a strict paradigm evaluation to fulfill the exam/theory requirements.

- **Faculty Conferral Schedule:**
    - The student will submit a brief weekly progress report via email detailing completed milestones.
    - Two brief (15-minute) virtual checkpoints will be requested: a mid-term check-in (Week 4) and a final project wrap-up review (Week 7) and mini appointments regarding questions.

5) Preparedness

As a senior Computer Science student, I have completed all foundational prerequisites. I have earned a grade of C or higher in both COS 280 (Discrete Mathematics) and COS 285 (Data Structures),and coursework much further, giving me the mathematical logic and algorithmic base necessary to study formal language criteria.
Beyond formal prerequisites, my capacity to successfully execute this accelerated study is verified by extensive practical engineering experience:

- **Advanced Systems and Architecture Experience:** Throughout my coursework and personal projects, I have gained experience designing and deploying asynchronous data pipelines and full-stack software systems using multi layer architectures, relational databases, Docker, and asynchronous task processing. I believe this background provides a solid foundation for understanding concepts related to language compilation, parsing, and execution, allowing me to focus more deeply on the core ideas behind programming language implementation.
- **Python as a Foundation for Studying Functional Programming:** Much of my recent technical work has been built in Python, and I am comfortable with its imperative and object-oriented programming paradigms. Because of this familiarity, I propose using Python as the platform for exploring functional programming concepts. This approach would allow me to concentrate on understanding functional design principles, side-effect management, and language trade-offs without simultaneously learning an entirely new language syntax, making better use of the project's seven-week timeline.
- **Independent Learning and Execution:** Through previous software engineering and machine learning projects, I have regularly worked from technical documentation, research papers, and academic resources to design and implement complete solutions with minimal supervision. These experiences have strengthened my ability to independently research unfamiliar topics, evaluate different approaches, and deliver meaningful technical work within a structured timeframe. I believe this experience will help me successfully carry out the proposed independent study.